

Correção das atividades da Semana 8

Semana 8 aula 2

Manipulação de texto

"""

Você está construindo um sistema de gerenciamento de contatos. Crie um programa que realize as seguintes tarefas:

a) Peça ao usuário para digitar seu nome completo.

b) Concatene “Olá,” com o nome fornecido e exiba o resultado.

c) Conte quantos caracteres existem no nome completo digitado e exiba o resultado.

d) Peça ao usuário para digitar um sobrenome para procurar na string do nome completo.

e) Verifique se o sobrenome fornecido está presente no nome completo e exiba uma mensagem apropriada.

f) Exiba o nome completo em letras maiúsculas.

g) Substitua todas as ocorrências da vogal “a” na string do nome completo pelo caractere ‘4’ e exiba o resultado.

"""

```
print("Sistema de gerenciamento de contatos")
```

```
# a) Solicita o nome completo do usuário
```

```
nome_completo = input("Digite seu nome completo: ")
```

```
# b) Concatena “Olá,” com o nome e exibe
```

```
print("Olá, " + nome_completo)
```

```
# c) Conta e exibe o número de caracteres no nome completo
```

```
total_caracteres = len(nome_completo)
```

```
print(f"Seu nome completo tem {total_caracteres} caracteres (contando espaços).")
```

```
# d) Solicita um sobrenome para procurar no nome completo
```

```
sobrenome = input("Digite um sobrenome para procurar no nome completo: ")
```

```
# e) Verifica se o sobrenome está presente e exibe mensagem apropriada
```

```
if sobrenome.lower() in nome_completo.lower():
```

```
    print(f"O sobrenome '{sobrenome}' está presente no nome completo.")
```

```
else:
```

```
    print(f"O sobrenome '{sobrenome}' **não** foi encontrado no nome completo.")
```

```
# f) Exibe o nome completo em letras maiúsculas
```

```
print("Nome completo em maiúsculas:", nome_completo.upper())
```

```
# g) Substitui todas as ocorrências da letra 'a' por '4'
```

```
nome_modificado = nome_completo.replace("a", "4").replace("A", "4")
```

```
print("Nome com 'a' substituído por '4':", nome_modificado)
```

Semana 8 aula 2

Exercício 01

"""

Crie duas variáveis do tipo string, uma contendo “Olá” e outra contendo “Mundo”. Concatene-as e imprima o resultado.

"""

```
parte1 = "Olá"
parte2 = "Mundo"
```

```
# Concatenando com espaço entre as palavras
mensagem = parte1 + " " + parte2
```

```
print(mensagem)
```

Exercício 02

"""

Crie duas variáveis do tipo string, uma contendo “Olá” e outra contendo “Mundo”. Concatene-as e imprima o resultado.

"""

```
texto = "Python"
print(texto[2])
```

Exercício 03

"""

Crie uma string qualquer. Substitua um dos caracteres por outro e imprima a nova string resultante.

"""

```
# Criando uma string
texto = "Python"
```

```
# Substituindo o caractere 'y' por 'a'
nova_string = texto.replace("y", "a")
```

```
# Imprimindo a nova string
print(nova_string)
```

Exercício 04

"""

Solicite ao usuário que digite seu nome. Em seguida, imprima o comprimento do nome fornecido.

"""

Solicita o nome ao usuário

nome = input("Digite seu nome: ")

Imprime o comprimento do nome

print("O comprimento do seu nome é:", len(nome))

Exercício 05

"""

Crie uma string que represente uma frase. Verifique se a palavra “gato” está presente na frase e imprima o resultado (verdadeiro ou falso).

"""

Criando uma string que representa uma frase

frase = "O cachorro e o gato estão brincando."

Verificando se a palavra "gato" está presente na frase

contém_gato = "gato" in frase

print(contém_gato)

Exercício 06

"""

Peça ao usuário que digite uma frase. Conte o número de palavras na frase e imprima o resultado.

"""

frase = input("Digite uma frase: ")

"""

Conta o número de palavras na frase. O método split() divide a frase em uma lista de palavras

"""

palavras = frase.split()

print("Número de palavras na frase:", len(palavras))

Exercício 07

"""

Crie uma função que receba uma frase como parâmetro e retorne a mesma frase com as palavras invertidas. Por exemplo, “Olá Mundo” deve ser transformado em “Mundo Olá”.

"""

```
def inverter_palavras(frase):
    # Divide a frase em uma lista de palavras
    palavras = frase.split()

    # Inverte a ordem das palavras
    palavras_invertidas = palavras[::-1]

    # Junta as palavras invertidas de volta em uma string
    frase_invertida = ' '.join(palavras_invertidas)

    return frase_invertida

frase = input("Digite uma frase: ")

# Função
resultado = inverter_palavras(frase)
print("Frase com as palavras invertidas:", resultado)
```

Exercício 08

"""

Solicite ao usuário que digite uma string que contenha espaços em branco no início e no final. Remova esses espaços e imprima a string resultante.

"""

```
texto = input("Digite uma string com espaços no início e no final: ")

# Remove os espaços em branco no início e no final usando strip()
texto_sem_espacos = texto.strip()

print("String sem espaços:", texto_sem_espacos)
```

Exercício 09

"""

Crie uma função que receba um número inteiro e retorne uma string que o represente com separadores de milhares. Por exemplo, para o número 1234567, a função deve retornar “1,234,567”.

"""

```
def formatar_com_separador_milhares(numero):  
    # Utiliza a função format() para adicionar separadores de milhares  
    return "{:,}".format(numero)
```

```
numero = int(input("Digite um número inteiro: "))
```

```
# Chama a função  
resultado = formatar_com_separador_milhares(numero)  
print(f"Número formatado: {resultado}")
```

Exercício 10

""" *Implemente uma função que receba uma string e um número (chamado de “deslocamento”) como parâmetros e retorne a string cifrada, usando a Cifra de César. Cada letra na string deve ser deslocada pelo número fornecido. Por exemplo, com um deslocamento de 3, “ABC” seria cifrado como “DEF”.* """

```
def cifra_cesar(texto, deslocamento):  
    resultado = ""  
  
    for char in texto:  
        # Verifica se o caractere é uma letra  
        if char.isalpha():  
            # Determina o valor base (para letras minúsculas ou maiúsculas)  
            base = ord('a') if char.islower() else ord('A')  
  
            # Calcula o novo caractere com o deslocamento  
            novo_char = chr(base + (ord(char) - base + deslocamento) % 26)  
            resultado += novo_char  
        else:  
            # Se não for uma letra, apenas adiciona o caractere sem modificar  
            resultado += char  
  
    return resultado
```

```
texto = input("Digite o texto para cifrar: ")  
deslocamento = int(input("Digite o número de deslocamento: "))
```

Função

```
texto_cifrado = cifra_cesar(texto, deslocamento)
print("Texto cifrado:", texto_cifrado)
```

"""

ord() converte uma letra para seu código ASCII (valor numérico correspondente).

chr() converte um código ASCII de volta para o caractere.

O deslocamento é aplicado ao código ASCII da letra, e a operação % 26 garante que o deslocamento ocorra de forma cíclica no alfabeto (de A a Z ou de a a z).

Caracteres não alfabéticos (como espaços, pontuações) são mantidos inalterados.

"""

Exercício 11

"""

Escreva um programa que receba uma palavra ou frase do usuário e determine se ela é um palíndromo ou não. O programa deve ignorar maiúsculas e minúsculas, bem como espaços em branco.

Um palíndromo é uma sequência de caracteres, como uma palavra, frase ou número, que permanece a mesma quando lida de trás para frente. Isso ocorre devido à simetria na disposição dos caracteres.

"""

```
def verificar_palindromo(texto):
    # Remove espaços em branco e converte para minúsculas
    texto = texto.replace(" ", "").lower()

    # Verifica se a string é igual à sua reversa
    return texto == texto[::-1]
```

```
# Solicita uma palavra ou frase ao usuário
texto_usuario = input("Digite uma palavra ou frase: ")
```

```
# Verifica se é um palíndromo e imprime o resultado
if verificar_palindromo(texto_usuario):
    print("É um palíndromo!")
else:
    print("Não é um palíndromo!")
```